Greedy algorithms

Part IV Advanced Design and Analysis Techniques

Scope

A greedy algorithm retains only one choice per step, avoiding other candidates. Activity exchanges, Huffman leaf exchanges, and matroid augmentation prove when a choice is safe. Weighted objectives and capacity restrictions show when an old rule fails.

Prerequisites

Sorting, exchange arguments, trees, and axiomatic selection properties.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Prove activity selection by exchange.
- Distinguish greedy choice from optimal substructure.
- Build Huffman codes and prove matroid greedy correctness.
- Give counterexamples when safety fails and complete every exchange step.
- 16.1 An activity-selection problem
- 16.2 Elements of the greedy strategy
- 16.3 Huffman codes
- 16.4 Matroids and greedy methods
- 16.5 A task-scheduling problem as a matroid

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Activity selection

Given all start/finish times, maximize the number of pairwise nonoverlapping activities. Assume s_i<f_i, sorted nondecreasing finishes, and half-open intervals[s_i,f_i). Equal finish/start endpoints are compatible.

Theorem. For every nonempty subproblem, some maximum compatible set contains an earliest-finishing activity a_m.

Proof. Let A* be an optimum and a_j its earliest-finishing activity. Since f_m≤f_j, replace a_j with a_m.

- Every later activity in A* starts at or after f_j, hence also after f_m.
- There is no earlier activity in the subproblem's chosen schedule to disrupt.
- Cardinality is unchanged, so the new set remains optimal.

This establishes a safe greedy choice. After a_m, only activities starting at or after f_m remain. Repeating the same exchange inductively proves optimality.

Linear scan and boundaries

```
activity-select(activities sorted by finish):
    result=[]; last_finish=-∞
    for (s,f) in activities:
        if s>=last_finish:
            append (s,f) to result
            last_finish=f
    return result
```

Before processing activity j, result is the compatible set chosen by the rule from the first j−1 activities, and last_finish is its final endpoint. The exchange theorem makes each choice extendible to an optimum. One scan takes Θ(n) when sorted; otherwise add O(n log n) sorting.

Assume positive-duration half-open intervals; equal endpoints may touch. Zero-duration activities need separate same-point rules. Empty input returns an empty set. Earliest start is unsafe: [0,10), [1,2),[2,3) gives it one activity rather than two.

Shortest duration is also unsafe: [0,3),[3,6),[2,4). Choosing the middle length2 activity blocks both length3 activities and gives one rather than the optimum two.

Complete activity scan

Activity	Start	Finish
a	1	3
b	2	5
c	3	6
d	5	7
e	7	9
f	8	10

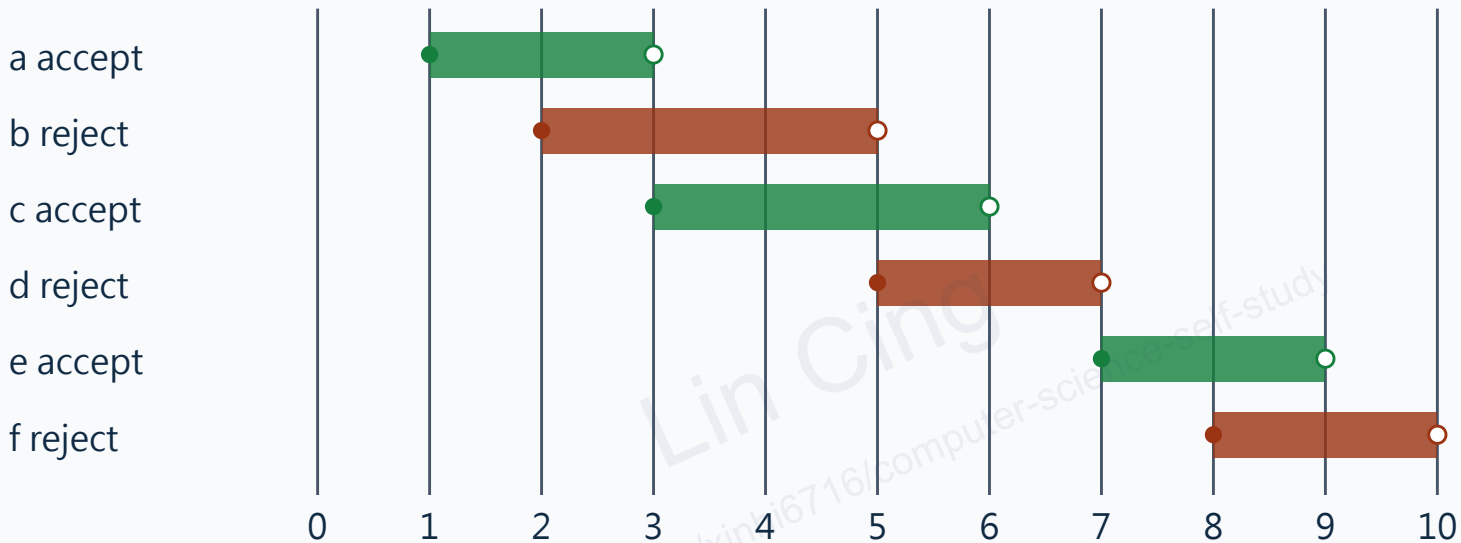
Select a; reject b since2<3; select c since3≥3; reject d since5<6; select e since7≥6; reject f since8<9. Result{a,c,e}.

The optimum need not be unique: {b,d,e} also has size3. Exchange guarantees **some** optimum includes the earliest finish, not that every optimum must.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete activity scan (continued)

Earliest-finish scan selects a, c, e



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elements of the greedy strategy

An exchange proof must establish:

1. **Safety.** For greedy g and an optimum A*, identify a differing choice x and form A′ by replacing x with g. Check feasibility and no loss of value.
2. **The residual problem.** After selecting g, the remaining solution belongs to a precisely defined subproblem that retains every relevant candidate.
3. **Termination by induction.** Empty/singleton bases are correct and each decision strictly shrinks the problem.

Only then do safe local choices imply global optimality. Immediate benefit, average performance, or successful examples alone do not suffice.

Largest-coin greedy fails for denominations{1,3,4}, amount6:4+1+1 uses three coins versus3+3 using two. This disproves the general rule; experience with familiar denominations does not prove a property for all denomination families.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elements of the greedy strategy (continued)

A safe local choice belongs to some global optimum

Activities

Optimal a_j — Earliest a_m

Replace by a_m; retain later choices.

Huffman

Deepest a,b — Min x,y

Exchange never raises cost.

Matroid

$A \in I$ — add $b \in B \setminus A$

Augment and remain independent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why weighted activities invalidate the same exchange

Maximizing total value changes the objective from cardinality.

Activity	Interval	Value
short	[0,2)	1
long	[1,3)	100
tail	[3,4)	1

Earliest finish selects short+tail, value2; optimum long+tail has101. Replacing long by short preserves compatibility but lowers value100→1, so it no longer preserves optimality.

The weighted problem still has optimal substructure. Sort by finish with one-based indices. Let p(j) be the largest earlier index finishing no later than s_j, or0 if none. D[j] is optimum value among the first j activities, with D[0]=0:

`D[j]=max(D[j-1],value_j+D[p(j)]).`

Why weighted activities invalidate the same exchange (continued)

Excluding j leaves the first j−1 activities. Including it restricts all remaining choices to the first p(j), which can be optimized independently. These cases cover every solution. Here p=(0,0,2),D=(0,1,100,101). Reconstruct tail at D3 and long at D2. Finish-time greedy guarantees unweighted cardinality only.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fractional and 0–1 knapsack require different proofs

Assume positive weights/values and $C>0$. Fractional knapsack allows $x_i \in [0,1]$, maximizing $\Sigma v_i x_i$. Sort decreasing density $\rho_i = v_i/w_i$, taking highest densities fully and at most one partial item.

If total weight $\leq C$, taking everything is optimal. Otherwise any optimum fills C: spare space plus a partially untaken positive-value item would allow improvement.

Suppose an optimum uses positive lower-density j weight while higher-density i is not full. Move $\delta = \min(w_i(1-x_i), w_j x_j) > 0$ weight from j to i. Capacity is unchanged and value increases by $\delta(\rho_i - \rho_j) \geq 0$. Repeat until the first greedy choice agrees, then induct on remaining capacity. Some optimum follows the whole density order; ties may be arbitrary.

In 0–1 knapsack, $x_i \in \{0,1\}$, so that δ exchange is usually illegal. For $C=10$ and $(w,v) = (6,12),(5,9),(5,9)$, density greedy takes $(6,12)$ and cannot fill the remaining 4, earning 12. The 0–1 optimum takes both $(5,9)$, earning 18. Fractional greedy takes $(6,12)$ and $4/5$ of one $(5,9)$, earning 19.2.

For 0–1 integer-capacity DP, $D[i,c]$ is optimum for the first i items and capacity at most c. Bases $D[0,c] = D[i,0] = 0$. Take max of $D[i-1,c]$ and $v_i + D[i-1, c-w_i]$, the latter only if $w_i \leq c$. Time $\Theta(nC)$ is pseudopolynomial because C occupies only $\Theta(\log C)$ input bits.

Huffman codes

A binary prefix tree's leaf depth is code length. For frequency f_c, total bits are

`B(T)=Σ_c f_c d_T(c).`

Repeatedly extract the two minimum-weight trees x,y and insert their parent of weight f_x+f_y. n symbols need n−1 rounds, each two extract-min and one insert, or3(n−1) heap operations. With linear heap construction, time is O(n log n).

Swapping left/right changes codeword0/1 values but not lengths or total cost. Equal frequencies may yield different optimal trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman's safe choice places minimum frequencies at deepest siblings

Assume at least two symbols and positive frequencies. An optimal prefix tree may be full: removing a one-child internal node shortens every code beneath it and lowers cost. A deepest leaf has a leaf sibling at the same depth, since an internal sibling would have deeper descendants.

Let x,y have the two lowest frequencies. First move x to one of these deepest positions. If one holds y, use the other, keeping y available for the second exchange or already in its desired sibling position.

Swapping x with deepest leaf a changes only their contributions:

$B(T')-B(T)=(f_a-f_x)(d_x-d_a)\leq 0$.

Here $f_a \geq f_x$ and $d_x \leq d_a$. With x fixed, all remaining frequencies are at least f_y. Move y to x's deepest sibling without raising cost or moving x. A symbol already in place gives a zero exchange; equal frequencies also permit ties.

Thus some optimal prefix code includes this greedy choice. The argument requires both the minimum-frequency and maximum-depth inequalities, not intuition alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman contraction and optimal substructure

Replace x,y by virtual symbol z of frequency f_z=f_x+f_y. Expand z in an optimum contracted tree T′ into leaves x,y to obtain T.

Other leaf depths stay unchanged, while x,y have depth d_T′(z)+1. Hence

$B(T)=B(T')+f_x+f_y$.

If a better original tree T* existed, the safe-choice lemma permits deepest siblings x,y. Contracting them would give

$B(T^{**})=B(T^*)-f_x-f_y<B(T')$,

contradicting optimality of T′. Expansion is optimal, and induction on alphabet size proves Huffman.

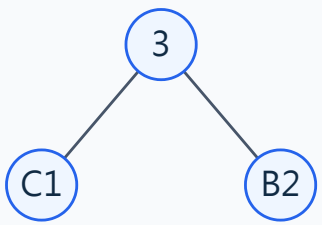
For A5,B2,C1,D3, merge weights are3,6,11. One code is A=0,D=10,C=110,B=111, costing5+6+3+6=20=3+6+11.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Huffman contraction and optimal substructure (continued)

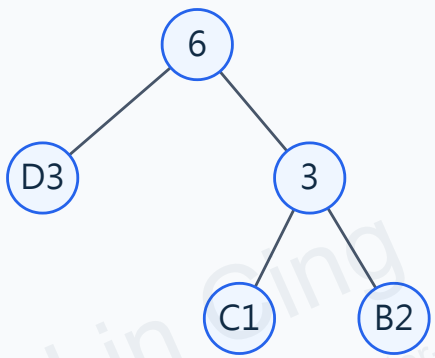
Huffman merges the two lightest trees each time

C1+B2 → 3



Q weights:1,2,3,5 →3,3,5

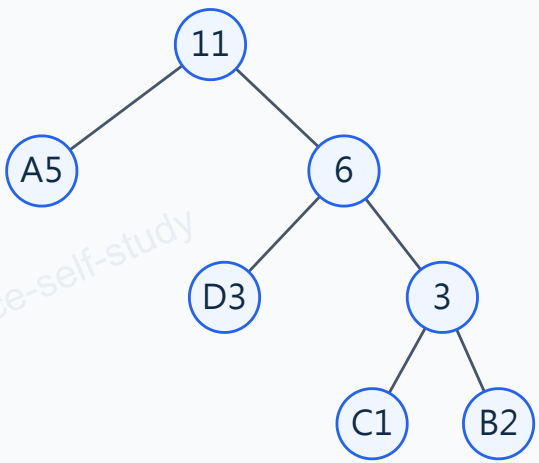
D3+3 → 6



Q weights:3,3,5 →5,6

Left0, right1

A5+6 → 11



Q weights:5,6 →11

Codes A=0, D=10, C=110, B=111

Cost = sum of merge weights3+6+11=20

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman algorithm and singleton boundary

```
Q = min-heap of all positive-frequency singleton trees
while size(Q)>1:
    x=extract_min(Q); y=extract_min(Q)
    insert(Q, node(x.weight+y.weight, x, y))
return Q's sole tree, or empty if Q is empty
```

Decode from root, following left/right for0/1. At a leaf, emit its symbol and restart at root. Prefix freedom means no complete word is a prefix of another, making boundaries unambiguous. Ending at an internal node means truncated input, not successful decoding.

An empty alphabet has no tree. A singleton may receive an empty code theoretically; practical streams often prescribe0 and transmit a symbol count separately. The equality between weighted length and merge-weight sum uses tree depths and, for the usual nontrivial coding case, at least two leaves. Do not substitute a singleton's forced one-bit implementation convention into merge cost.

Matroids and greedy methods

A matroid $M=(S,I)$ satisfies:

- Empty set belongs to I.
- Heredity: every subset of an independent set is independent.
- Augmentation: if $A,B\in I$ and $|A|<|B|$, some x in B excluding A makes $A\cup\{x\}$ independent.

Assume finite S and $w(x)>0$. Scan decreasing weights and accept if independence remains. Let x be the first independent singleton. If none exists, heredity excludes every nonempty independent set, and empty is optimal.

Take optimum B. If x is absent, start $A=\{x\}$. While $|A|<|B|$, augment using an element of B outside A. Only B elements are added. At equal size, exactly one $y\in B$ has been omitted, so $A=B-\{y\}+\{x\}$. Since $\{y\}$ is independent, first feasible x has $w(x)\geq w(y)$, and $w(A)=w(B)-w(y)+w(x)\geq w(B)$. Thus A is also optimal and contains x.

Do not assert that any arbitrary y can be exchanged. Augmentation first constructs feasible A, and equal cardinality identifies its one omitted y.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete proof that contraction remains a matroid

Fix feasible x, let S' be S excluding x, and $I' = \{A \subseteq S' : A \cup \{x\} \in I\}$. S' may contain elements that are never independent in the new system; these cause no problem.

- Empty belongs to I' because $\{x\} \in I$.
- If $A \in I'$ and $C \subseteq A$, then $C \cup \{x\} \subseteq A \cup \{x\}$; heredity gives $C \in I'$.
- If $A, B \in I'$ with $|A| < |B|$, augment $A \cup \{x\}$ from $B \cup \{x\}$. The added y lies in B excluding A, since x is in both. Hence $A \cup \{y\} \in I'$.

Removing x maps every feasible solution containing x to I' and subtracts exactly $w(x)$; adding it reverses this mapping. After choosing x, the original problem is exactly maximum-weight independence in the contraction. Induct on $|S|$; later greedy feasibility tests are precisely the I' oracle, proving optimality.

An element rejected now never becomes feasible later: heredity of a later larger feasible set containing it would imply feasibility at rejection, a contradiction. Sorting costs $O(n \log n)$; $O(f(n))$ per oracle gives $O(n \log n + n f(n))$ total. For arbitrary independent sets with negative weights, skip negative elements rather than unconditionally forming a full basis.

Graphic matroids and a nonmatroid counterexample

For fixed undirected G=(V,E), take S=E and I all acyclic edge sets, retaining isolated vertices in V. Heredity holds. Let A's components be V1..Vk. If every B edge stays within a component Vi, B's forest restriction there has at most|Vi|−1 edges. Summing gives|B|≤|V|−k=|A|, contradicting|B|>|A|. Thus B has an edge crossing A-components, and adding it creates no cycle. Augmentation holds.

In a connected graph, bases are n−1-edge spanning trees. To derive minimum-cost MST from the positive maximum-weight theorem, set w′(e)=K−cost(e)>0. Equal basis sizes make maximizing Σw′ equivalent to minimizing original cost. This precisely connects increasing-cost Kruskal to the theorem without confusing arbitrary minimum forests with bases.

Heredity alone does not imply augmentation. For S={a,b,c},weights10,9,9, take I={∅,{a},{b},{c},{b,c}}. A={a},B={b,c} have unequal sizes, yet neither b nor c augments A. Greedy gets10 at a; optimum{b,c} gets18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unit-time scheduling as a matroid

Job j takes one unit, has deadline d_j, and late penalty w_j. A set A can all finish on time exactly when sorted deadlines satisfy d_(k)≥k, equivalently for every t,

$$N_t(A)=|\{j \text{ in } A:d_j<=t\}|<=t.$$

Necessity follows from capacity t in the first t slots. For sufficiency, schedule by increasing deadline: job k's deadline≥k makes it on time.

All jobs are ready at0, nonpreemptive, on one machine. Deadlines are positive integers and completion exactly at d_j is on time. n jobs need only n slots, so cap deadlines above n at n without changing feasibility.

Heredity holds. For augmentation with|A|<|B|, let k be the largest integer with N_k(A)=k, including0. Since|A|<|B|≤n, k<n. A has exactly k jobs with deadline≤k, while B has at most k. Thus B has at least|B|−k>|A|−k jobs with deadline>k. Some x∈B outside A has d_x>k.

Adding x leaves N_t unchanged for t≤k. For t>k, maximality implies N_t(A)<t, so integrality gives N_t(A)≤t−1. Adding at most1 preserves feasibility, proving augmentation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete scheduling greedy trace

Job	Deadline	Penalty
a	3	9
b	1	8
c	3	7
d	2	6
e	1	5

Scan decreasing penalty: accept a; accept b with deadlines1,3; accept c with1,3,3. Reject d:1,2,3,3 has fourth deadline3<4. Reject e because N1=2>1.

On-time set{a,b,c} is scheduled b,a,c at completion times1,2,3. Place d,e at4,5; both are indeed late. Penalty is6+5=11. Total penalty35 minus maximum saved24 gives11.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete scheduling greedy trace (continued)

A length-n deadline-frequency array yields all prefix N_t and checks independence in O(n). n tests cost $O(n^2)$, plus O(n log n) initial penalty sorting. Sorting separately for every oracle costs O(n log n) each and $O(n^2 \log n)$ overall; sorting is not linear.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter16, §§16.1–16.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study